Telco-Retrieve

Retrieval-Augmented Generation for Telecom Specifications

Technical Guidebook

Open Telco AI · GSMA

Part of the Telco Studio Toolkit

Covers ingestion, chunking, enrichment, embedding, hybrid retrieval, reranking, generation, and evaluation — every configurable option exposed by the `open-telco-rag` CLI and library.

September 9, 2026

Contents

1 Introduction **5**
- 1.1 What is Telco-Retrieve? . . . 5
- 1.2 Architecture Overview . . . 5
- 1.3 Supported Corpora . . . 6

2 Installation & Quick Start **7**
- 2.1 Prerequisites . . . 7
- 2.2 Installation . . . 7
- 2.3 Environment Configuration . . . 7
- 2.4 Your First Ingest and Query . . . 7

3 Pipeline Components **9**
- 3.1 Source . . . 9
- 3.2 Chunker . . . 9
- 3.3 Embedder . . . 9
- 3.4 VectorStore . . . 10
- 3.5 Reranker . . . 10
- 3.6 Generator . . . 10

4 Chunking Strategies **11**
- 4.1 `text_baseline` . . . 11
- 4.2 `sliding_window_tokens` . . . 11
- 4.3 `parent_child` . . . 11
- 4.4 `hierarchical_markdown` . . . 11
- 4.5 `lumber_chunker` . . . 12

5 Enrichment Strategies **13**
- 5.1 `none` . . . 13
- 5.2 `metadata_tagging` . . . 13
- 5.3 `acronym_expansion` . . . 13
- 5.4 `llm_metadata` . . . 13

6 Embedding & Reranking Models **15**
- 6.1 Embedding Models . . . 15
- 6.2 Reranker Models . . . 15

7 Retrieval **17**
- 7.1 Search Methods . . . 17
- 7.2 Parent-Child Resolution . . . 17
- 7.3 Distance Metrics . . . 17

8 CLI Reference **19**
8.1 `ingest` . . . 19
8.2 `search` . . . 19
8.3 `ask` . . . 20
8.4 `evaluate` . . . 20
8.5 `info` . . . 21
8.6 `delete` . . . 21
8.7 `push-hf` / `pull-hf` . . . 22

9 Evaluation **23**
9.1 Design Philosophy . . . 23
9.2 Metric Reference . . . 24
9.3 Deterministic mc/tf Scoring . . . 24
9.4 Efficiency Tracking . . . 25
9.5 Ground Truth Question Format . . . 25

10 Python API Reference **27**
10.1 Ingestion . . . 27
10.2 Search and Generation . . . 27
10.3 Evaluation . . . 27

11 Performance & Cost Guide **29**
11.1 Embedding Batch Sizing . . . 29
11.2 Concurrency Tuning (LLM-Backed Stages) . . . 29
11.3 Reranking Cost . . . 29
11.4 Ingestion at Scale . . . 29

12 Best Practices **31**
12.1 Choosing a Chunking Strategy . . . 31
12.2 Choosing an Embedding Model . . . 31
12.3 Enabling Reranking . . . 31

13 Glossary **33**

A Related Tools **35**

Chapter 1

Introduction

1.1 What is Telco-Retrieve?

Telco-Retrieve (package name `open-telco-rag`) is a **modular Retrieval-Augmented Generation pipeline** built specifically for telecommunications standards. It ingests specification documents from seven standards bodies, indexes them for both dense semantic and sparse keyword search, and answers natural-language questions grounded in the retrieved text — with every stage independently swappable.

Core Value Proposition

General-purpose LLMs hallucinate telecom-specific detail, and fine-tuning on specifications freezes knowledge at training time. Telco-Retrieve instead retrieves the relevant specification text at query time and grounds the answer in it — current, traceable to a source document, and never dependent on a model having memorised a clause number correctly.

1.2 Architecture Overview

Telco-Retrieve is organised as a straight-line pipeline of independently replaceable stages, each defined as a Python `Protocol` rather than a concrete class hierarchy — swapping the embedding model, vector store, or generator backend never requires touching any other stage.

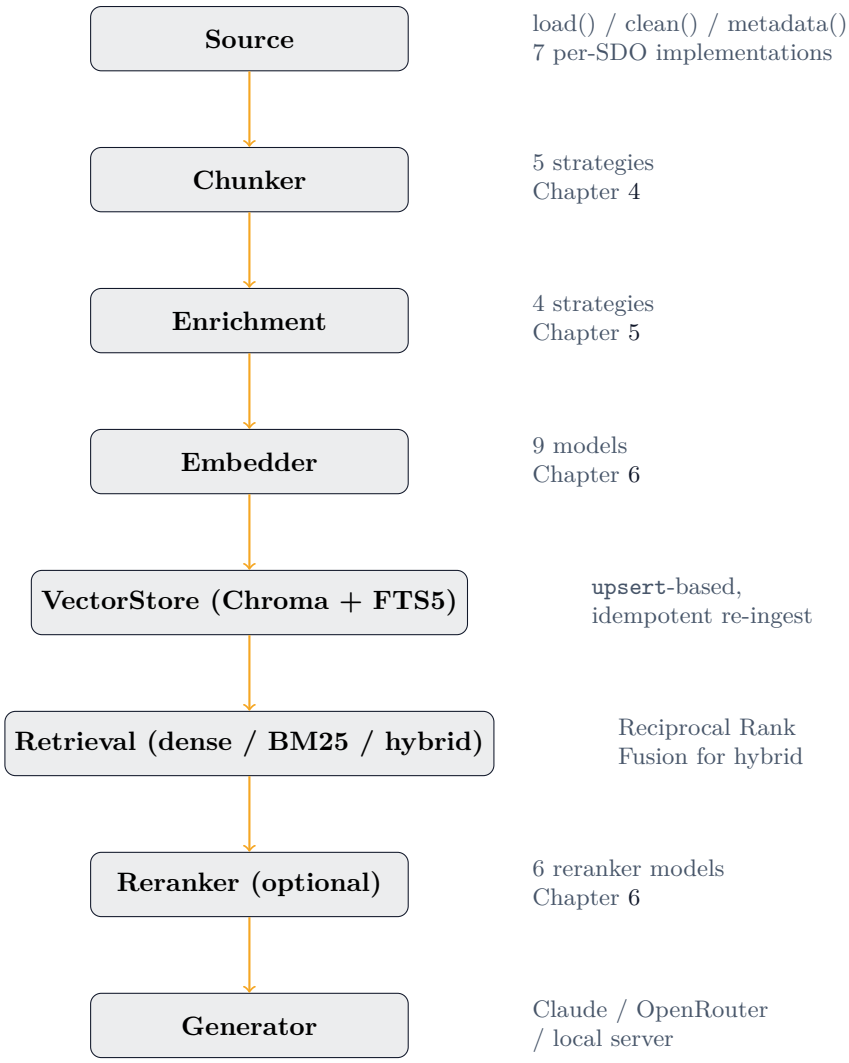


Figure 1.1: Telco-Retrieve pipeline stages. Ingest runs Source → Chunker → Enrichment → Embedder → VectorStore; query runs Retrieval → Reranker (optional) → Generator.

Note

Unlike TelcoSem's YAML-declared block flows, Telco-Retrieve's stage sequence is fixed (ingest always runs Source→Chunker→Enrichment→Embedder→ VectorStore in that order); what's configurable at each stage is *which implementation* runs, selected via CLI flags and the `configs/*.json` registries covered throughout this guide — not the sequence itself.

1.3 Supported Corpora

SDO	Key	Focus Areas	Chunks
O-RAN Alliance	`oran`	Near-RT RIC, E2/A1/O1, xApp, KPM	169,130
3GPP	`3gpp`	5G NR, 5GC, SBA, network slicing	2,229,734
ETSI	`etsi`	ZSM, ENI, MEC, NFV, SAI	984,728
ITU-T	`itu`	Y.3170 ML series, IMT-2020	575,762
TM Forum	`tmforum`	Open APIs, ODA, order-to-activate	6,905
CAMARA	`camara`	Open Gateway APIs, QoD, DeviceLocation	3,919
GSMA	`gsma`	security, SGP	121,118

Chapter 2

Installation & Quick Start

2.1 Prerequisites

- Python 3.10 or later
- At least one generator backend credential: `ANTHROPIC_API_KEY`, `OPENROUTER_API_KEY`, or a self-hosted OpenAI-compatible server
- Optionally `DATALAB_API_KEY` for higher-quality PDF conversion via the Marker API

2.2 Installation

```
git clone https://github.com/Znbne/telco-retrieve.git
cd telco-retrieve
pip install -e ".[local-embed,pdf,docx,dev]"

# Optional extras
pip install -e ".[cloud-embed]"   # OpenRouter-hosted embedding models
pip install -e ".[all]"           # everything
```

Note

`local-embed` pulls in `sentence-transformers` for locally-run embedding/reranker models; skip it if you only intend to use OpenRouter-hosted models (`-embed-backend openrouter`).

2.3 Environment Configuration

Create a `.env` file in the project root:

```
ANTHROPIC_API_KEY=your_key_here
OPENROUTER_API_KEY=your_key_here
DATALAB_API_KEY=your_key_here

# Self-hosted OpenAI-compatible server (vLLM/Ollama) --
# used by --generator local, --llm-backend local
LOCAL_LLM_API_KEY=your_vllm_server_api_key_here
LOCAL_LLM_BASE_URL=http://localhost:8000/v1
LOCAL_LLM_MODEL=Qwen/Qwen2.5-32B-Instruct
```

2.4 Your First Ingest and Query

```
# Ingest O-RAN specs with the fast development-baseline embedder
open-telco-rag ingest oran --data data/oran --model minilm

# Ask a question against what you just ingested
open-telco-rag ask "What interfaces does the Near-RT RIC expose?" \
  --source-type oran --model minilm --generator openrouter

# Or pull a pre-built database instead of ingesting from scratch
open-telco-rag pull-hf GSMA/telco-retrieve-chunks \
  3gpp-r19_bge_text_baseline_none_c1024_o200 --local-dir dbs
open-telco-rag ask "What triggers a conditional handover?" --source-type 3gpp \
  --model bge --db-path dbs/3gpp-r19_bge_text_baseline_none_c1024_o200 \
  --collection 3gpp-r19_bge_text_baseline_none_c1024_o200 --generator openrouter
```

Tip

Run `open-telco-rag info -db-path <path>` at any point to list every collection at that path along with its chunk count — use the collection name it prints for `-collection` on `search`/`ask`.

Note

The pre-built databases on `GSMA/telco-retrieve-chunks` are a 120-configuration grid over the 3GPP Release-19 corpus — every combination of embedding model, chunking strategy, and enrichment strategy in this guide. Folder names encode the configuration: `3gpp-r19_<model>_<chunking>_<enrichment>_<params>`. Each is a complete ChromaDB + SQLite-FTS5 index; pull one and query it with no ingestion or GPU.

Chapter 3

Pipeline Components

Every stage in Figure 1.1 is a Python `Protocol` defined once in `open_telco_rag/types.py`. This chapter summarises each Protocol's contract; Chapters 4–7 cover the implementations available for each one in detail.

3.1 Source

```python
class Source(Protocol):
    def load(self, path) -> str: ...
    def clean(self, text: str) -> str: ...
    def metadata(self, path) -> dict: ...
    def supported_extensions(self) -> list[str]: ...
```

Seven implementations, one per SDO, each handling that source's own file formats and filename/folder conventions:

Class	Formats	Notes
`ORANSource`	PDF, DOCX	Optional Marker API conversion
`TPPSource`	Markdown	TSpec-LLM dataset; two filename layouts
`CAMARASource`	YAML, Markdown	YAML-to-prose conversion before chunking
`TMForumSource`	DOCX, PDF, JSON, YAML	Six-pattern repo classifier
`GSMASource`	PDF, DOCX	Optional Marker API conversion
`ITUSource`	PDF	Optional Marker API conversion
`ETSISource`	PDF, YAML	Folder-first doc-type/ISG classification

3.2 Chunker

```python
class Chunker(Protocol):
    def chunk(self, text: str) -> list[Chunk]: ...
```

Dispatched dynamically via `chunk_document()` based on `-strategy`; see Chapter 4 for all five.

3.3 Embedder

```python
class Embedder(Protocol):
    def embed(self, texts: list[str]) -> list[list[float]]: ...
    @property
    def model_key(self) -> str: ...
```

Two implementations: `SentenceTransformerEmbedder` (local) and `OpenRouterEmbedder` (hosted); nine registered models between them — see Chapter 6.

3.4 VectorStore

```python
class VectorStore(Protocol):
    def add(self, ids, embeddings, documents, metadatas) -> None: ...
    def search(self, embedding, n_results: int = 5) -> list[SearchResult]: ...
    def count(self) -> int: ...
    def exists(self, id: str) -> bool: ...
    def all_doc_ids(self) -> list[str]: ...
```

`ChromaStore` is the only implementation, backed by ChromaDB with `upsert`-based writes (safe to re-run ingest on already-processed files). A parallel SQLite FTS5 index is maintained alongside every collection for BM25 search — see Chapter 7.

3.5 Reranker

```python
class Reranker(Protocol):
    def rerank(self, query: str, results: list[SearchResult],
               top_k: int) -> list[SearchResult]: ...
```

`CrossEncoderReranker` (three general-purpose models) or `OTelReranker` (three telecom-tuned Qwen3-based models) — see Chapter 6.

3.6 Generator

```python
class Generator(Protocol):
    def generate(self, question: str,
                 results: list[SearchResult]) -> dict: ...
```

Three implementations: `ClaudeGenerator` (Anthropic API), `OpenRouterGenerator` (any OpenRouter-hosted model), and a local generator talking to any OpenAI-compatible self-hosted server. Every generator is instructed to answer only from the provided context and to say so explicitly when the context is insufficient, rather than guessing.

Chapter 4

Chunking Strategies

Selected via `-strategy`, dispatched by `chunk_document()` in `open_telco_rag/core/chunking.py`. Structural bounds for each strategy are read from `configs/chunking_strategies.json`, not hardcoded, so `-chunk-size`/`-overlap` are validated against the strategy actually selected.

Key	Unit	Default size	Default overlap	LLM?
`text_baseline`	characters	1024	200	No
`sliding_window_tokens`	tokens	512	50	No
`parent_child`	characters	2048	0	No
`hierarchical_markdown`	characters	4000	0	No
`lumber_chunker`	characters	1500	0	Yes

4.1 `text_baseline`

Generic paragraph-based splitting with a sliding character-window overlap. The fallback for raw PDF extraction (via `pypdf`) that carries no Markdown structure to split on. Overlap ensures a concept split across a paragraph boundary still appears complete in at least one chunk.

4.2 `sliding_window_tokens`

Fixed-size segments tokenized against `cl100k_base` via `tiktoken`, with an explicit token-count overlap rather than a character-count one — useful when chunk size needs to track a model's actual context window rather than an approximate character count.

4.3 `parent_child`

Two-tier chunking: large parent segments are split first, then each parent is further split into small child chunks that carry the full parent text in their own metadata. Retrieval can then match on the precise child chunk while the generator receives the full parent context — enabled at query time via `-resolve-parents`.

4.4 `hierarchical_markdown`

Splits on Markdown heading boundaries (`#` through `#####`) first, falling back to paragraph aggregation only when a section exceeds the configured maximum size. Applied natively to Markdown/YAML sources and to PDFs preprocessed through the Marker API, since both carry real heading structure to split on.

4.5 `lumber_chunker`

Note

Requires an LLM connection: `-llm-backend local|openrouter` plus `-chunking-model`. A pre-flight connectivity check runs once before ingestion starts (not per-file), so a misconfigured endpoint fails immediately with a clear message instead of every worker process discovering it independently.

Uses an LLM's own structural comprehension to identify logical topic boundaries directly, rather than relying on formatting cues — useful for sources with weak or absent heading structure where the fixed strategies above would otherwise split mid-concept.

Chapter 5

Enrichment Strategies

Selected via `-enrichment`, dispatched by `enrich_chunks()` in `open_telco_rag/core/enrichment.py`, applied after chunking and before embedding.

5.1 none

Chunks pass through unmodified. The baseline for comparison against every other strategy.

5.2 metadata_tagging

Writes extracted metadata (SDO, work group, document type, etc.) into both the chunk's stored ChromaDB metadata *and* a text header prepended to the chunk before embedding (e.g. `[Document Context -> WG: WG3]`) — so the same information is available for structured filtering and for the embedding model to see directly.

5.3 acronym_expansion

Checks chunk text against a fixed telecom-acronym dictionary (`RAN`, `RIC`, `O-RAN`, `3GPP`, and others), matched on word boundaries, and appends an `[Acronym Expansions]` block listing every match found — zero LLM cost, purely dictionary-driven.

5.4 llm_metadata

Note

Requires an LLM connection, same as `lumber_chunker` — shares the same `-llm-backend`/`-llm-base-url` connection and pre-flight check when both are used together in one ingest run.

Uses an LLM to extract domain-specific abbreviations and technical terms directly from each chunk's own content (not a fixed dictionary), formatted as a JSON array and prepended as a `Technical Context & Definitions` header. Catches terminology the fixed `acronym_expansion` dictionary doesn't cover, at the cost of one LLM call per chunk.

Chapter 6

Embedding & Reranking Models

6.1 Embedding Models

Selected via `-model`, registered in `configs/embedding_models.json`.

Key	Model	Dims	Notes
`minilm`	all-MiniLM-L6-v2	384	Fast, lightweight; development baseline
`mpnet`	all-mpnet-base-v2	768	Stronger general-purpose
`e5`	intfloat/e5-large-v2	1024	High quality, retrieval-optimised
`bge`	BAAI/bge-large-en-v1.5	1024	Retrieval-optimised, strong on technical domains
`otel-109m`	OTel-Embedding-109M	768	Telecom-tuned, 109M params
`otel-300m`	OTel-Embedding-300M	1024	Telecom-tuned, 300M params (recommended default)
`otel-0.6b`	OTel-Embedding-0.6B	1024	Telecom-tuned, highest-quality OTel embedder
`or-3-small`	openai/text-embedding-3-small	1536	OpenRouter-hosted
`or-bge-m3`	baai/bge-m3	1024	OpenRouter-hosted

Tip

`-embed-backend local` (default) runs `minilm`/`mpnet`/`e5`/`bge`/`otel-*` via `sentence-transformers`. `-embed-backend openrouter` is required for `or-3-small` and `or-bge-m3`, and can also run any of the local-capable models remotely if you'd rather not download weights.

6.2 Reranker Models

Selected via `-reranker-model`, registered in `configs/reranker_models.json`. Enabled with `-rerank`, which retrieves 3×`top_k` candidates and reranks down to `-rerank-top-k`.

Key	Backend	Notes
`ms-marco-minilm`	cross-encoder	Fast, lightweight baseline (default)
`ms-marco-large`	cross-encoder	Larger, better accuracy, slower
`bge-reranker`	cross-encoder	Retrieval-optimised, strong on technical domains
`otel-reranker-0.6b`	otel (Qwen3)	Telecom-tuned, 0.6B params (recommended)
`otel-reranker-4b`	otel (Qwen3)	Telecom-tuned, 4B params, higher accuracy
`otel-reranker-8b`	otel (Qwen3)	Telecom-tuned, 8B params, highest accuracy

Chapter 7

Retrieval

7.1 Search Methods

Selected via `-search-method` on `search`/`ask`, and via `evaluate`'s equivalent retrieval path.

Value	Mechanism	Best for
`dense`	ChromaDB nearest-neighbour on the embedding	Default; general semantic queries
`bm25`	SQLite FTS5 keyword match	Exact clause numbers, parameter names
`hybrid`	Both, fused via Reciprocal Rank Fusion	Best of both; slight latency cost

$$\text{RRF_Score}(d) = \sum_{m \in \{\text{dense},\text{bm25}\}} \frac{1}{k + r_m(d)} \tag{7.1}$$

where $r_m(d)$ is document d's rank within method m's result list and k is a fixed smoothing constant. Fusing by rank rather than raw score avoids having to normalise dense cosine distance against sparse BM25 scores, which live on entirely different scales.

7.2 Parent-Child Resolution

`-resolve-parents` expands retrieved child chunks (from `parent_child` chunking) into their full parent text before generation, without an extra store lookup — the parent text is already carried in the child's own metadata at ingestion time.

7.3 Distance Metrics

Set once at ingestion via `-distance-metric` (`l2` default, or `cosine`, `ip`), configuring the collection's HNSW index. `SearchResult.similarity` converts a cosine-space distance d via $1 - d$; the evaluation harness (see Chapter 9) uses its own e^{-d} conversion instead, which stays well-behaved regardless of which distance metric a given collection was actually built with.

Chapter 8

CLI Reference

Every subcommand below is a full options reference — not one specific configuration, but every currently-integrated flag and its default. Flags common to several subcommands (`-model`, `-db-path`, `-collection`) behave identically wherever they appear.

8.1 `ingest`

```
open-telco-rag ingest <source_type> [options]
```

`source_type`: one of `oran`, `3gpp`, `camara`, `tmforum`, `gsma`, `itu`, `etsi`.

Flag	Default	Description
`-data`	`data/{source_type}`	Data directory
`-model`	`minilm`	Embedding model key
`-db-path`	`chroma_db`	ChromaDB path
`-collection`	`{source_type}_{model}`	Override collection nam
`-chunk-size`	1024	Chunk size (unit `-strategy`)
`-overlap`	200	Chunk overlap
`-strategy`	`text_baseline`	Chunking strategy (Ch
`-enrichment`	`none`	Enrichment strategy (C
`-enrichment-model`	`google/qwen-2.5-coder-32b`	Model for `llm_metada`
`-distance-metric`	`l2`	`l2` / `cosine` / `ip`
`-embed-backend`	`local`	`local` / `openrouter`
`-batch-size`	256 (local) / 500 (openrouter)	Chunks per embedding
`-use-marker`	off	PDF conversion via Da API (needs `DATALAB_A`
`-llm-backend`	`local`	Backend `lumber_chunker/llm_`
`-llm-base-url`	env `LOCAL_LLM_BASE_URL` or `http://localhost:8000/v1`	Local LLM server URL
`-chunking-model`	env `LOCAL_LLM_MODEL` or backend default	Model for `lumber_chu`
`-no-resume`	off	Disable resume-from-p

8.2 `search`

```
open-telco-rag search "<query>" [options]
```

Flag	Default	Description
`-source-type`	`oran`	Source type
`-model`	`minilm`	Embedding model key
`-db-path`	`chroma_db`	ChromaDB path
`-collection`	`{source_type}_{model}`	Override collection name
`-top-k`	5	Results to return
`-search-method`	`dense`	`dense` / `bm25` / `hybrid`
`-rerank`	off	Enable reranking
`-rerank-top-k`	5	Results kept after reranking
`-reranker-model`	`ms-marco-minilm`	Reranker key (Chapter 6)
`-resolve-parents`	off	Expand child chunks to full parent context
`-embed-backend`	`local`	`local` / `openrouter`

8.3 ask

```
open-telco-rag ask "<question>" [options]
```

`ask` runs `search` and then generates a grounded answer from the retrieved chunks in one step. It accepts every retrieval flag `search` does (default `-top-k` is 8, not 5), plus generator selection:

Flag	Default	Description
`-generator`	`openrouter`	`claude` / `openrouter` / LLM backend for the an
`-generator-model`	backend default	Model id/key: OpenRou local served-model nam model
`-llm-base-url`	env `LOCAL_LLM_BASE_URL` or `http://localhost:8000/v1`	Server URL for `-genera`
`-llm-api-key`	env `LOCAL_LLM_API_KEY`	Bearer token for `-gener`

```
# OpenRouter-hosted model
open-telco-rag ask "..." --source-type 3gpp --model bge --collection <name> \
  --generator openrouter --generator-model meta-llama/llama-3.3-70b-instruct

# Self-hosted OpenAI-compatible server (vLLM, Ollama, ...)
open-telco-rag ask "..." --source-type 3gpp --model bge --collection <name> \
  --generator local --generator-model Qwen/Qwen3-32B \
  --llm-base-url http://localhost:8000/v1
```

Note

`-generator claude` needs `ANTHROPIC_API_KEY`; `-generator openrouter` needs `OPENROUTER_API_KEY`. The local and OpenRouter backends are fully interchangeable — the same question and retrieval settings, only the answering model differs.

8.4 evaluate

```
open-telco-rag evaluate <corpus|all> [options]
```

`corpus`: one of the seven source keys, or `all` to run every corpus sequentially.

Flag	Default	Descriptio
`-model`	`minilm`	Embedding
`-db-path`	`chroma_db`	ChromaDB
`-collection`	`{corpus}_{model}`	Override co
`-questions-file`	`configs/eval_questions/{corpus}.json`	Custom qu
`-top-k`	5	Chunks ret
`-threshold`	0.5	Similarity tio
`-generator`	`openrouter`	LLM provi
`-generator-model`	backend default	Model for
`-rag-generator`	same as `-generator`	LLM provi evaluated
`-rag-generator-model`	same as `-generator-model`	Model for ated
`-llm-base-url` / `-llm-api-key`	env vars	For `-gener`
`-rag-llm-base-url` / `-rag-llm-api-key`	falls back to the `-llm-*` pair	For `-rag-g`
`-rerank` / `-rerank-top-k` / `-reranker-model`	off / 5 / `ms-marco-minilm`	Same as `se`
`-include-baseline`	off	Also run a pass for co
`-inject-metadata`	off	Prepend ca to the ques query)
`-include-keywords`	off	Append qu search quer
`-resolve-parents`	off	Expand ch context
`-output`	`results`	Directory f
`-no-save`	off	Print summ
`-leaderboard`	`results/leaderboard.json`	Leaderboar
`-no-leaderboard`	off	Skip the le
`-run-name`	none	Custom lab
`-delay`	1.0	Seconds be iting)
`-debug`	off	Log raw L metric call

8.5 `info`

```
open-telco-rag info [--db-path chroma_db] [--collection NAME]
```

Lists every collection at `-db-path` (or just the one named by `-collection`) with its chunk count.

8.6 `delete`

```
open-telco-rag delete <collection> [--db-path chroma_db]
```

Drops a ChromaDB collection and its paired FTS5 index together.

8.7 `push-hf` / `pull-hf`

```
open-telco-rag push-hf <db_path> <repo_id> [--path-in-repo NAME] [--public]
open-telco-rag pull-hf <repo_id> <path_in_repo> [--local-dir ./runs]
```

Sync a whole Chroma+FTS database folder to/from a Hugging Face dataset repository. Repos are private by default; `-public` on `push-hf` opts out.

Chapter 9

Evaluation

Note

Generating and reviewing the question sets `evaluate` reads is **TelcoQnA**'s job, not Telco-Retrieve's — see the TelcoQnA guidebook for the generation pipeline, question formats, and human review workflow. This chapter covers only what `open-telco-rag evaluate` itself computes and how to configure it; it deliberately does not describe any one particular evaluation run.

9.1 Design Philosophy

Retrieval quality and generation quality are measured independently. A wrong final answer can originate in either stage, and conflating them into one score makes it impossible to tell which component to fix.

9.2 Metric Reference

Metric	Stage	Ground truth?	What it measures
Cosine similarity stats	Retrieval	No	Mean/min/max similarity of retrieved chunks
Distinct sources retrieved	Retrieval	No	Number of distinct documents contributing chunks
Coverage ratio	Retrieval	No	Fraction of chunks above `-threshold`
Canonical source hit	Retrieval	No†	Expected (release-qualified) document retrieved?
Exact chunk hit	Retrieval	No†	The exact source chunk retrieved?
Faithfulness	Generation	No	Fraction of answer claims grounded in retrieved chunks
Answer relevancy	Generation	No	Does the answer address the question's intent
Context recall	Retrieval	Yes	Fraction of reference statements found in retrieved chunks
Answer correctness	Generation	Yes	Factual F1 + semantic similarity vs. reference
Accuracy (mc/tf)	Generation	Yes	Exact-match verdict, correct/incorrect/abstained breakdown

† Uses the source/chunk recorded on the question at generation time, not a separately-authored reference answer.

Tip

`-threshold`'s default (0.5) clears close to 100% of retrieved chunks in most model/chunking combinations observed so far — a weak discriminator at that setting. Calibrate it per embedding model before trusting coverage ratio for cross-run comparison.

9.3 Deterministic mc/tf Scoring

Multiple-choice and true/false questions are scored deterministically via a schema-constrained JSON verdict (with a regex-based prose fallback for backends that don't honour the schema), rather than by an LLM judge. Three numbers are reported, not one:

- `mean_accuracy` — correct / total mc+tf questions
- `mean_accuracy_attempted` — correct / (correct + incorrect), abstentions excluded
- `abstention_rate` — abstained / total mc+tf questions

A composite `weighted_score_pct` combines mc/tf and open-ended questions into one comparable figure (mc/tf: 1 point per correct verdict; open/fill-blank: 5 points when answer correctness clears 0.7), so corpora with a different question-format mix stay comparable.

9.4 Efficiency Tracking

Every question also records latency (search/rerank/generation phases separately), token usage (prompt/completion/cached/reasoning, falling back to a `tiktoken` estimate when a backend doesn't report usage), and cost where the backend exposes pricing. Judge/metric-call token usage is tracked separately from RAG-generation usage, since judge cost is roughly constant across a model/chunking sweep and shouldn't dilute a comparison meant to isolate what changed. Every run appends one row — full configuration plus the full metric set above — to `results/leaderboard.json` for cross-run comparison.

9.5 Ground Truth Question Format

```
// configs/eval_questions/oran.json
{
  "questions": [
    {
      "question":          "What is the role of the Conflict Manager?",
      "answer_type":        "open",
      "keywords":           ["Conflict Manager", "xApp", "priority"],
      "corpus":             "oran",
      "canonical_source":   "O-RAN WG3 E2GAP Clause 5.1",
      "reference_answer":   "The Conflict Manager microservice...",
      "chunk_id":           "O-RAN.WG3.E2GAP::minilm::0",
      "release":            ""
    }
  ]
}
```

Populated via TelcoQnA's generation-and-review pipeline, or hand-authored following the same schema.

Chapter 10

Python API Reference

10.1 Ingestion

```python
from open_telco_rag.core.pipeline import ingest
from open_telco_rag.sources.oran import ORANSource
from open_telco_rag.embedders.sentense_transformers import SentenceTransformerEmbedder
from open_telco_rag.stores.chroma import ChromaStore

source   = ORANSource()
embedder = SentenceTransformerEmbedder(model_key="minilm")
store    = ChromaStore(path="chroma_db", collection_name="oran_minilm")

stats = ingest(
    source=source, store=store, embedder=embedder,
    data_dir="data/oran", strategy="hierarchical_markdown",
    enrichment="metadata_tagging",
)
print(f"{stats.chunks_stored} chunks stored, {stats.error_files} errors")
```

10.2 Search and Generation

```python
from open_telco_rag.core.pipeline import search, generate

results = search(
    "What is the E2 interface?",
    store=store, embedder=embedder, top_k=5,
    search_method="hybrid",
)
answer = generate("What is the E2 interface?", results, source_type="oran")
print(answer["answer"])
print(answer["sources"])
```

10.3 Evaluation

```python
from open_telco_rag.evaluation.evaluate import evaluate_corpus
from open_telco_rag.evaluation.questions import load_questions
from open_telco_rag.evaluation._claude import make_llm_call

questions = load_questions("oran")
llm_call  = make_llm_call("openrouter", "meta-llama/llama-3.3-70b-instruct")
```

```
result = evaluate_corpus(
    corpus="oran", store=store, embedder=embedder,
    questions=questions, llm_call=llm_call, top_k=8,
)
print(f"weighted_score_pct: {result.weighted_score_pct}")
```

Chapter 11

Performance & Cost Guide

11.1 Embedding Batch Sizing

Backend	Default `-batch-size`	Notes
Local (`sentence-transformers`)	256	Matches typical GPU micro-batch size
OpenRouter	500	Network round-trip dominates; larger batches am

11.2 Concurrency Tuning (LLM-Backed Stages)

Applies to `lumber_chunker` chunking, `llm_metadata` enrichment, and `evaluate`'s judge calls.

Backend	Guidance	Notes
OpenRouter (free tier)	Conservative	Rate limits vary by model
OpenRouter (paid)	Higher throughput	Check model-specific rate limits
Self-hosted vLLM/Ollama	Limited by GPU memory	Tune per your own hardware

11.3 Reranking Cost

`-rerank` retrieves 3×`top_k` candidates before reranking down to `-rerank-top-k` — reranking cost scales with that larger candidate count, not the final result count. A larger `-rerank-top-k` relative to `-top-k` narrows the initial retrieval's safety margin; keep the 3× ratio in mind when tuning both.

11.4 Ingestion at Scale

Ingestion is a two-phase design: Phase 1 (parsing/chunking) uses a `ProcessPoolExecutor` across CPU cores; Phase 2 (embedding + storage) uses GPU-sized batches and a small thread pool for concurrent Chroma/FTS5 writes. A resume file tracks completed files, so a large ingestion run across thousands of documents can be safely interrupted and restarted without reprocessing what already finished.

Chapter 12

Best Practices

12.1 Choosing a Chunking Strategy

1. **Prefer `hierarchical_markdown`** for any source with real heading structure (Markdown, YAML, or Marker-converted PDF) — it preserves section context that flat chunking discards.
2. **Use `text_baseline`** only as a fallback for raw PDF extraction with no structure to split on.
3. **Consider `parent_child`** when precise child-level retrieval matters but the generator needs broader context — pair with `-resolve-parents`.
4. **Reserve `lumber_chunker`** for sources where structural cues genuinely fail; it costs one LLM call per boundary decision.

12.2 Choosing an Embedding Model

1. **Start with `minilm`** for fast local development iteration.
2. **Move to `otel-300m`** (the recommended telecom-tuned default) once validating real retrieval quality on telecom-specific vocabulary.
3. **Reach for `otel-0.6b` or `bge`** when retrieval precision matters more than ingestion/query speed.
4. **Use the OpenRouter models** only when avoiding local GPU/CPU load is the priority — they carry network latency per embedding call.

12.3 Enabling Reranking

Reranking helps most when initial retrieval is noisy (broad queries, large corpora with overlapping vocabulary across documents) and helps least when retrieval is already precise (narrow, well-localised queries). Measure before and after on your own corpus rather than assuming it always helps — it always costs additional inference time.

Chapter 13

Glossary

Canonical source
: The release-qualified document identifier a question was written against (e.g. `"25.401 (Rel-19)"`), used by canonical source hit scoring.

Chunk
: A fixed-size (or heading-bounded) text segment extracted from a specification document.

Coverage ratio
: Fraction of retrieved chunks above a similarity threshold; needs per-model calibration.

Distance metric
: The vector-space geometry (`l2`/`cosine`/`ip`) a ChromaDB collection's HNSW index was built with.

Enrichment
: A pre-embedding text/metadata augmentation step (Chapter 5).

Hybrid search
: Dense + BM25 retrieval fused by Reciprocal Rank Fusion.

Protocol
: A Python structural-typing interface — any class implementing the right methods satisfies it, with no inheritance required.

Reranker
: A cross-encoder that re-scores an initial candidate set for higher precision at the top of the result list.

SDO
: Standards Development Organisation (O-RAN, 3GPP, ETSI, ITU-T, TM Forum, CAMARA, GSMA).

Weighted score pct
: A composite accuracy figure combining mc/tf and open/fill-blank questions into one comparable number.

Appendix A

Related Tools

`telco-qna-generation` (TelcoQnA)
: Synthetic QnA dataset generation and human review, producing the question sets this pipeline evaluates against.

`telcosem` (TelcoSem)
: Knowledge graph construction and GraphRAG retrieval, a complementary retrieval strategy to this pipeline's flat chunk retrieval.

GSMA/telco-retrieve-chunks
: HuggingFace dataset of pre-built retrieval databases — a 120-configuration grid (embedding model × chunking strategy × enrichment strategy) over the 3GPP Release-19 corpus, each a complete ChromaDB + SQLite-FTS5 index. Pull one with `pull-hf` and query it without ingesting. https://huggingface.co/datasets/GSMA/telco-retrieve-chunks

GSMA/telco-retrieve-QnA
: HuggingFace dataset of expert-validated 3GPP question–answer pairs (multiple-choice, true/false, open-ended) — the ground-truth question sets `evaluate` scores against. https://huggingface.co/datasets/GSMA/telco-retrieve-QnA